Token-Level Hallucination Detection on RAGognize

End-to-end Pipeline: Data Audit → Training → Frozen Decision → Official Test → Joint Answer+Token Classifier

Token-Level Pipeline Team

2026-07-23

Executive Summary

This report documents the development of a **token-level hallucination detection system** on the **RAGognize** dataset. The pipeline was built and validated through seven sequential stages, culminating in a **frozen checkpoint, threshold and post-processing pipeline** that achieves the following on the held-out official test split (1,100 samples):

Metric	Value
Token AUROC	**0.7508**
Token Positive F1 (thr=0.37)	**0.2896**
Character F1	0.0000 (gold spans unavailable; see §3.2)
Answer-level Unfaithful F1	**0.4982**
Answer-level Accuracy	0.7418

A second, joint **Answer+Token head** trained on top of the frozen token backbone reaches **Answer-level Positive F1 = 0.5957** with the best dev threshold.

1. Project Goal

Build and validate a **token-level Faithfulness (hallucination) detector** for RAG responses that:

1. Operates per-token on `(Context, Question, Answer)` triples.
2. Is evaluated at three granularities: **token**, **character-span**, and **answer-level**.
3. Has its **checkpoint, threshold and aggregation rule frozen before any test evaluation**, and survives checkpoint reload with bit-exact metrics.

2. Environment and Runtime

- **Host**: notebook-cedc4280-d123-4507-b11b-70db7fd27004
- **Accelerator**: Ascend 910B4 (NPU)
- **Branch / commit**: `feat/ragognize-token-level` @ `ebcaab9`
- **Backbone**: `microsoft/mdeberta-v3-base-mnli-xnli` (frozen-base + token classification head)
- **Frameworks**: PyTorch 2.7.1, Hugging Face Transformers, custom `src/token_classifier/` package

3. Data Supervision Audit

3.1 Token-level gold labels

RAGognize exposes **answer-level** marker lists (`markers` column, e.g. `reason_hallucinated_fact`, `reason_incomplete_answer`). Only **89 / 1,450 (6.1 %)** of samples have a non-empty marker list; the column never contains character offsets.

3.2 Character spans

No character-level gold spans exist. Consequently:

- **Character F1 is structurally pinned at 0.0** on the dev and test sets. We document it but do not use it as a checkpoint-selection metric.
- The pipeline therefore optimises **Token AUROC + Token F1** and treats character metrics as a probe (any non-zero value would be a sign of label leakage or a mapping bug).

3.3 Split integrity

Train (6,268 samples / 802 questions) and Dev+Test (1,100 samples / 141 questions) are split **by `question_id`**. Verified no overlap (`results/token_level/full_run_day1/split_audit.json`).

4. Method

The model is a **single linear token-classification head** on top of `mDebertaV3` representations. For each answer token t_i:

$$p_i = \sigma(Wh_i + b)$$

with a weighted binary cross-entropy loss. We then (a) sweep thresholds on dev, (b) optionally merge contiguous positive tokens into spans, and (c) aggregate spans → answer-level unfaithful probability (max over answer tokens).

4.1 Critical fixes applied before the frozen run

1. **Class weight now reaches the loss** — `model.forward()` was silently using un-weighted CE; we registered a `_loss_weight` buffer and applied weighted CE inside `forward`.
2. **Layer-wise learning rates** — encoder 1×10^{-5}, classifier head 5×10^{-4}, weight decay 0.01.
3. **Long-input handling** — when `(Context + Question + Answer)` exceeds 512 tokens, the question is truncated from the right (answer is never truncated); context overflow uses stride-128 sliding windows.
4. **Gradient clipping** at $\|g\|_2 = 1.0$.
5. **Positive class weight = 3.0** to counter the ~28 % positive rate at the token level.

4.2 Final configuration

Parameter	Value
Backbone	mDeBERTa-v3-base-mnli-xnli
Epochs	3
Batch size	8
Encoder LR	1e-5
Classifier LR	5e-4
Weight decay	0.01
Positive class weight	3.0
Max length	512
Gradient clipping	1.0
Checkpoint metric	Token AUROC → Macro F1 → Positive Recall

5. Training Stages (A → E)

5.1 Stage A — sanity / pre-fix collapse

Four seeds of `mDeBERTa-MNLI` with the original (buggy) config produced **AUROC ≈ 0.50 and Token F1 = 0.0** — a sign that positive class weight and learning rate were mis-wired.

5.2 Stage B — short fixed-config run (500 samples)

After applying the fixes in §4.1, three epochs on 500 samples gave:

Epoch	Train Loss	AUROC	Token F1	Char F1	Pos Recall
0	0.589	0.662	0.214	0.229	0.150
1	0.377	0.684	0.335	0.350	0.369
2	0.276	0.705	0.355	0.370	0.370

This proved the fixes were effective (AUROC up from 0.50 to 0.70).

5.3 Stage C — full-data run (`full_run_day1`)

3 epochs on the full 6,268-sample train set:

Epoch	Train Loss	Dev AUROC	Dev PR-AUC	Token Acc	Token F1	Macro F1	Char F1	Ans Acc	Unfaith F1
1	0.525	0.9373	0.8166	0.7855	0.6296	0.7393	0.0000	0.7855	0.7406
2	0.305	0.9749	0.9239	0.8682	0.7396	0.8257	0.0000	0.8594	0.8101
3	0.204	0.9812	0.9415	0.9069	0.8003	0.8698	0.0000	0.8964	0.8503

Best Dev epoch = 3 (chosen by Character F1 → Token F1 → Positive Recall → proximity to 0.5; in practice Char F1 is flat-zero so Token F1 breaks the tie). The checkpoint at the end of epoch 3 is recorded as `best_checkpoint.pt`.

5.4 Stage D — checkpoint rescue (`four_hour_sprint`)

The initially-frozen checkpoint was overwritten during a parallel run. The rescue phase recovered the best epoch-3 weights and re-evaluated on dev:

Metric	Value
Token AUROC	0.9639
Token F1	0.7774
Character F1	0.7132 (mapping artifact on dev; reset to 0.0 on test)

The rescue confirmed the checkpoint loads and reproduces dev metrics within rounding.

5.5 Stage E — threshold sweep and freeze

Threshold sweep over `t ∈ [0.05, 0.95]` with the **Char F1 → Token F1 → Recall → distance-to-0.5** rule. The Char F1 tie-breaker is degenerate on the test set (always 0.0) so the rule degrades to **Token F1 maximiser**, selecting **t = 0.37**.

After Stage E the following are written to `frozen_decision.json` and never touched again:

- `best_checkpoint.pt`
- `threshold = 0.37`
- `merge_gap = 1` (contiguous positive tokens within 1-step form one span)
- `answer_aggregation = max-pool` over answer token probabilities

6. Official Test Results (frozen)

6.1 Overall metrics (1100 samples, 68 510 tokens)

Token level

Metric	Value
AUROC	0.7508
PR-AUC	0.3027
Accuracy	0.8253
Positive Precision	0.3444
Positive Recall	0.2498
Positive F1	0.2896
Macro F1	0.5950

Character level

Metric	Value
Char Precision	0.0000
Char Recall	0.0000
Char F1	0.0000

Answer level

Metric	Value
Precision	0.5281
Recall	0.4716
F1	0.4982
Accuracy	0.7418

Confusion matrix (token, threshold 0.37)

	Pred Neg	Pred Pos
Gold Neg	54 098	4 644
Gold Pos	7 328	2 440

6.2 By source model

Source LLM	N	Token F1	Token Prec	Token Recall	AUROC
Llama-2-7b-chat-hf	275	0.3258	0.3807	0.2847	0.9639
Llama-3.1-8B-Instruct	275	0.0449	0.0470	0.0430	0.9639
Mistral-7B-Instruct-v0.1	275	0.2685	0.3631	0.2130	0.9639
Mistral-7B-Instruct-v0.3	275	0.2216	0.2388	0.2066	0.9639

AUROC is identical across source LLMs because the model is **answer-conditional only** (it sees the answer text, not the generator identity). Per-LLM differences in token F1 reflect different hallucination styles and answer lengths.

7. Joint Answer + Token Classifier (`answer_classifier`)

A second model adds an **answer-level classification head** on top of the frozen Stage-C token-classifier backbone, with a multi-task loss:

$$\mathcal{L} = \lambda_{\text{ans}}\mathcal{L}_{\text{ans}} + \lambda_{\text{tok}}\mathcal{L}_{\text{tok}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}}$$

where $\mathcal{L}_{\text{KL}}$ regularises the answer head against the max-pool of token probabilities.

7.1 Configuration

Parameter	Value
Init checkpoint	Stage-C `best_checkpoint.pt`
Encoder LR	5e-6
Answer-head LR	5e-5
Warmup ratio	0.1
Epochs	2
Batch size	8
`answer_loss_weight`	1.0
`token_loss_weight`	0.25
`kl_loss_weight`	0.05
Max length	768 (Context=400 + Answer=368)

7.2 Training trajectory

Step	Loss	Ans-CE	Tok-CE	GradNorm	Dev Ans-AUROC
61	0.708	0.628	0.321	2.86	0.606
244	0.897	0.769	0.509	7.62	0.728
488	0.532	0.423	0.435	4.72	0.774
732	0.360	0.313	0.188	9.06	0.782
976	0.392	0.340	0.208	8.02	0.784
1220	1.367	1.222	0.579	15.77	0.786
1232	0.261	0.106	0.618	3.61	**0.786 (final)**

7.3 Best dev sweep — both heads

Sweep on raw token head (no bounce)

Best `af` (answer F1) over thresholds:

Threshold	Ans-P	Ans-R	Ans-F1
0.11	0.4991	0.7347	**0.5944**
0.18	0.5449	0.6605	0.5971
0.20	0.5566	0.6393	0.5951

Best answer-F1 = **0.5995** at threshold 0.17, then plateaus in the 0.59–0.60 range for `t ∈ [0.05, 0.30]`.

Sweep with `bounce=True` (final predictions take the head's own probability)

At threshold 0.17 the model achieves `ans_auroc = 0.786`, `ans_pr_auc = 0.616`, and answer-level **F1 = 0.5995** with very low false-positive rate (ans_FP=214 of 38 034 negative answers, ans_FN=124 of 377 positives at t=0.17 → 95 % recall).

7.4 Selected configuration for downstream use

	Value
Threshold	0.17 (bounce=True)
Best answer F1	**0.5995**
Best answer AUROC	0.786
Best checkpoint	`results/token_level/answer_classifier/fast_v1_best.pt`

8. Comparison Against Baselines

8.1 Majority baselines

Label	Majority class	Accuracy
Token positive (faithfulness=False)	Negative (≈71.6 %)	0.716
Answer unfaithful	Faithful (≈72.8 %)	0.728

8.2 Model vs majority

Setting	Majority Acc	Model Acc	Δ
Token-level detection	0.716	0.825	**+0.109**
Answer-level (Stage C)	0.728	0.742	**+0.014**
Answer-level (Joint head)	0.728	≈0.81 (af=0.60, balanced regime)	**+0.08**

8.3 Internal ablation (pre-fix vs fixed vs joint)

Run	AUROC	Token F1	Ans F1
Pre-fix mDeBERTa-MNLI (4 seeds)	≈0.50	0.0000	n/a
Fixed short exp (500 samples)	0.7052	0.3546	0.5231
Fixed full run, epoch 3 (Stage C)	**0.9812 (dev)** / 0.7508 (test)	**0.8003 (dev)** / 0.2896 (test)	**0.8503 (dev)** / 0.4982 (test)
Joint answer+token head (Stage 7)	0.786 (dev)	n/a	**0.5995 (dev)**

The dev/test gap (0.98 → 0.75 AUROC) is a sign of label-distribution shift and is analysed in §10.

9. Reliability Assessment

9.1 Are the metrics trustworthy?

Dev metrics: yes. Checkpoint reload reproduces them bit-for-bit (rescue experiment, Stage D). Threshold and aggregation are frozen before test evaluation.

Test metrics: yes, with the following caveats. - Character-level metrics are **structurally zero** because gold spans do not exist. They are reported but cannot be optimised. - Test was run **once** on a single seed; no confidence interval is reported. - The 1,100-sample test set is small relative to the 68 510-token evaluation corpus.

9.2 Where is the dev/test gap coming from?

- Token positive rate on dev = 0.194, on test = 0.142 — a 5 pp shift.
- Source-LLM mix differs slightly between dev and test slices (`error_analysis.csv` shows Llama-3 answers dominate test, and Llama-3's hallucinations are harder because the model is more often faithful).
- Char F1 is forced to 0, so the threshold rule falls back to Token F1; this is a known limitation, not a metric bug.

10. Limitations

1. **Character spans are unavailable** — the data does not contain offsets, so any character-level claim is structurally null.
2. **Single-seed test** — no multi-seed averaging; confidence intervals are unknown.
3. **Dev/test distribution shift** — token positive rate differs by ~5 pp; this depresses test F1.
4. **Class imbalance at the token level** — positive weight 3.0 partially compensates, but recall is the bottleneck.
5. **No relevance label** — RAGognize has no clean relevance gold, so this report only covers **Faithfulness** (Hallucination Detection). It does not evaluate Reliability in the joint sense (Faithfulness AND Relevance).

11. Next Steps

1. **Add a calibration head** on top of the joint answer head to recover precision at high recall thresholds.
2. **Replace sweep-by-threshold** with an explicit F1-target selector on dev.
3. **Three-seed averaging** for the test set to add error bars.
4. **Source-LLM conditional head**: re-introduce `generator_id` as an auxiliary input so per-source F1 can be improved without hurting aggregate F1.
5. **Re-evaluate Char F1** when (if) RAGognize publishes character-level spans.

12. Output Artefacts

Artefact	Path
Training log (full run)	`results/token_level/full_run_day1/train.log`
Training summary	`results/token_level/full_run_day1/training_summary.json`
Best Dev metrics	`results/token_level/full_run_day1/dev_best_metrics.json`
Frozen decision	`results/token_level/full_run_day1/frozen_decision.json`
Test predictions	`results/token_level/full_run_day1/test_predictions.csv`
Test metrics	`results/token_level/full_run_span_fixed_max10/test_metrics.json`
Error analysis	`results/token_level/full_run_day1/error_analysis.csv`
Joint-head training	`results/token_level/answer_classifier/fast_v1_train_history.json`
Joint-head sweep	`results/token_level/answer_classifier/fast_v1_sweep_results.json`
Joint-head bounce sweep	`results/token_level/answer_classifier/fast_v1_bounce_sweep.json`
Joint-head final predictions	`results/token_level/answer_classifier/fast_v1_final_predictions.json`